

Floating point numbers, binary numbers & random numbers

Niklas Kersten & Timo Weber

October 2025

b-adic representation of whole numbers

Let $b, n \in \mathbb{N}$ with $b \geq 2$, then there are uniquely determined numbers $l \in \mathbb{N}, z_i \in \{0, \dots, b-1\}$ for $i = 0, \dots, l-1$ and $z_{l-1} \neq 0$ such that

$$n = \sum_{i=0}^{l-1} z_i b^i.$$

b-adic representation of whole numbers

Let $b, n \in \mathbb{N}$ with $b \geq 2$, then there are uniquely determined numbers $l \in \mathbb{N}, z_i \in \{0, \dots, b-1\}$ for $i = 0, \dots, l-1$ and $z_{l-1} \neq 0$ such that

$$n = \sum_{i=0}^{l-1} z_i b^i.$$

$z_{l-1} \dots z_0$ is called *b-adic representation* of n and we write $n = (z_{l-1} \dots z_0)_b$.

b-adic representation of whole numbers

Let $b, n \in \mathbb{N}$ with $b \geq 2$, then there are uniquely determined numbers $l \in \mathbb{N}, z_i \in \{0, \dots, b-1\}$ for $i = 0, \dots, l-1$ and $z_{l-1} \neq 0$ such that

$$n = \sum_{i=0}^{l-1} z_i b^i.$$

$z_{l-1} \dots z_0$ is called *b-adic representation* of n and we write $n = (z_{l-1} \dots z_0)_b$.
For $b = 2$ we get the *binary* numbers, consisting of $l \in \mathbb{N}$ *bits*.

How do we represent whole numbers now?

Reserving the first bit for the sign, we can represent negative numbers aswell, consider the following example with 4 bits.

$$0000 = 0$$

$$0001 = 1$$

$$0010 = 2$$

$$0011 = 3$$

...

$$0111 = 7$$

$$1000 = -0$$

$$1001 = -1$$

$$1010 = -2$$

$$1011 = -3$$

...

$$1111 = -7$$

How do we represent whole numbers now?

Reserving the first bit for the sign, we can represent negative numbers as well, consider the following example with 4 bits.

$$0000 = 0$$

$$0001 = 1$$

$$0010 = 2$$

$$0011 = 3$$

...

$$0111 = 7$$

$$1000 = -0$$

$$1001 = -1$$

$$1010 = -2$$

$$1011 = -3$$

...

$$1111 = -7$$

Problem: We have two representations for 0 and addition or subtraction is more complicated, because we have to make case distinctions.

The b-complement

Idea: We can represent negative numbers complementarily. Again we reserve the first bit for the sign, but we represent the negative numbers $z \in \{-8, \dots, -1\}$ as $z + 2^4$.

The b-complement

Idea: We can represent negative numbers complementarily. Again we reserve the first bit for the sign, but we represent the negative numbers $z \in \{-8, \dots, -1\}$ as $z + 2^4$. Thus we get,

$$1000 = -8$$

$$1001 = -7$$

$$1010 = -6$$

...

$$1111 = -1.$$

The b-complement

For $b \in \mathbb{N}_{\geq 2}$, $l \in \mathbb{N}_{\geq 1}$ and $n \in \{0, 1, \dots, b^l - 1\}$ we define the *b-complement of n with length l* as

$$K_b^n(n) := -n \bmod b^l.$$

The b-complement

For $b \in \mathbb{N}_{\geq 2}$, $l \in \mathbb{N}_{\geq 1}$ and $n \in \{0, 1, \dots, b^l - 1\}$ we define the *b-complement of n with length l* as

$$K_b^n(n) := -n \bmod b^l.$$

A convenient way to compute the 2-complement:

For $n > 0$ take the 2-adic representation of $n - 1$ and 'flip' the remaining digits.

The floating point numbers

In computer arithmetic (finite arithmetic) we do not have all real numbers, but only a finite subset instead. We call this subset the *floating point numbers*. For a *base* $b \in \mathbb{N}_{\geq 2}$, a *precision* $l \in \mathbb{N}_{\geq 1}$ and an *exponent* e with $e_{\min} \leq e \leq e_{\max}$, a floating point number has the form

$$x = \pm b^e (d_1 b^{-1} + \dots + d_l b^{-l})$$

where $d_i \in \{0, \dots, b-1\}$ are the *digits*.

The floating point numbers

In computer arithmetic (finite arithmetic) we do not have all real numbers, but only a finite subset instead. We call this subset the *floating point numbers*. For a *base* $b \in \mathbb{N}_{\geq 2}$, a *precision* $l \in \mathbb{N}_{\geq 1}$ and an *exponent* e with $e_{\min} \leq e \leq e_{\max}$, a floating point number has the form

$$x = \pm b^e (d_1 b^{-1} + \dots + d_l b^{-l})$$

where $d_i \in \{0, \dots, b-1\}$ are the *digits*.

An alternative way to write x is $x = \pm d_1 d_2 \dots d_l \times b^e$.

The floating point numbers

We denote the set of these floating point numbers as

$$F(b, l, e_{min}, e_{max}) \subset \mathbb{R}.$$

The floating point numbers

We denote the set of these floating point numbers as

$$F(b, l, e_{min}, e_{max}) \subset \mathbb{R}.$$

Example:

$$F(2, 2, -1, 1) = \{0, \pm\frac{1}{4}, \pm\frac{3}{8}, \pm\frac{1}{2}, \pm\frac{3}{4}, \pm 1, \pm\frac{3}{2}\}$$

Limitations of floating point numbers

Since the set of floating point numbers is finite, every real number $x \in \mathbb{R} \setminus F(b, l, e_{min}, e_{max})$ has to be rounded.

Limitations of floating point numbers

Since the set of floating point numbers is finite, every real number $x \in \mathbb{R} \setminus F(b, l, e_{min}, e_{max})$ has to be rounded.

How can we still do sensible computations, that are transferable from one machine to another?

Most computer arithmetic uses IEEE 754 standard.

(IEEE = Institute of Electrical and Electronics Engineers)

- double precision: $F(2, 53, -1022, 1023)$
- single precision: $F(2, 24, -126, 127)$

In double precision we use 53 bits for the digits and 11 bits for the exponent, adding up to 64 bits.

In single precision we use 24 bits for the digits and 8 bits for the exponent, adding up to 32 bits.

Limitations of floating point numbers

Using floating point numbers to represent real numbers, will result in errors on a representation level and on an operational level

Representation Error

Example: Decimal 0.1 in IEEE 754 Single Precision

In binary, $0.1_{10} = 0.0001100110011 \dots_2$, which repeats infinitely.

Thus, the stored value must be rounded to:

$$\text{fl}(0.1) = 0.10000000149011612$$

This produces an error:

$$|x - \hat{x}| = |0.1 - 0.10000000149011612| = 1.49011612 \times 10^{-9}.$$

Even simple decimals cannot always be represented exactly in binary.

Error in floating point operations

For the same reasons, every operation on floating point numbers introduces an error

$$\begin{aligned}\text{fl}(x \circ y) &= (x \circ y)(1 + \delta), \\ |\delta| &\leq \varepsilon_{\text{mach}}.\end{aligned}$$

- 1 float32: $\varepsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$
- 2 float64: $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$

Rounding

In general there are several strategies for rounding in floating point operations

Guard-digits example

Subtract $10.1 - 9.93$ with precision $p = 3$ significant digits.

No guard:

$$x = 1.01 \times 10^1,$$

$$y = 0.99 \times 10^1$$

$$\Rightarrow x - y = 0.02 \times 10^1 = 0.2$$

(wrong; true is 0.17)

With one guard digit ($l+1$):

$$x = 1.010 \times 10^1,$$

$$y = 0.993 \times 10^1$$

$$\Rightarrow x - y = 0.017 \times 10^1 = 0.17$$

(exact)

The IEEE 754 not only clearly defines different rounding modes, but also

- Denormal representation of numbers with exponent field completely 0 as opposed to normalization, where the leading significand bit is implicitly 1. This allows gradual underflow.
- Encodings for $\pm\infty$, NaNs, and ± 0 .
- exception cases, e.g. $\sqrt{-1} \rightarrow \text{NaN}$

From Deterministic Representation to Randomness

- So far, we have seen that numbers in a computer are represented deterministically as a finite number of bits.
- Yet, much of scientific computing relies on randomness, e.g. modelling random movement of particles, random sampling, or simply whether you get picked to present in the next exercise.

From Deterministic Representation to Randomness

- So far we have seen that numbers in a computer are represented deterministically as a finite number of bits.
- Yet, much of scientific computing relies on randomness, e.g. modelling random movement of particles, random sampling, or simply whether you get picked to present in the next exercise.

How can a machine that behaves deterministically generate something that looks random?

Simulating Randomness

In computers:

- All inputs known $\Rightarrow$ Output reproducible
- Pseudorandomness
- Algorithms that seemingly create Outputs with no predictable pattern.

Programs that solve this task are referred to as Pseudorandom Number Generators (PRNG)

PRNG: Linear Congruential Generator

A simple example for a PRNG is the Linear Congruential Generator

Definition

$$X_{n+1} \equiv (aX_n + c) \bmod m$$

Example ($a=5$, $c=1$, $m=16$, $X_0=7$):

$7 \rightarrow 4 \rightarrow 5 \rightarrow 10 \rightarrow 3 \rightarrow 0 \rightarrow 1 \rightarrow 6 \rightarrow 15 \rightarrow 12 \rightarrow \dots$

The *seed* is what we use to initialize a PRNG, the *state* is the PRNGs full internal memory after initialization

PRNG: Mersenne Twister

A widely used PRNG for simulations, *not* an LCG.

- State: 624 **32-bit unsigned integers**.
- Period: $2^{19937} - 1$

. Deterministic seeding: same seed $\Rightarrow$ same initial state and sequence.